

Analyzing MongoDB Transactions

Name:

Kyle R. Cabrera

Introduction

This lab focuses on implementing MongoDB transactions to ensure data consistency. It demonstrates how multiple database operations can be executed as a single atomic unit. The objective is to understand how transactions help prevent partial updates and maintain integrity.

Procedure

1. Configured MongoDB as a replica set.
2. Created a sample database (bankDB) and accounts collection.
3. Connected MongoDB using Python (PyMongo).
4. Implemented a successful transaction transferring funds.
5. Simulated a failed transaction to test rollback.
6. Verified balances after each operation.

Results

Initial State:

Alice: 100

Bob: 50

After Successful Transaction:

Alice: 70

Bob: 80

After Failed Transaction (Rollback):

Alice: 100

Bob: 50

Output

Successful Transaction Output:

Transaction committed. Transfer successful.

Alice Balance: 70

Bob Balance: 80

Failed Transaction Output (Rollback):

Transaction aborted.

Simulated error during transaction

Alice Balance: 100

Bob Balance: 50

Analysis

In this lab, I implemented MongoDB transactions to perform a fund transfer between two accounts. I encountered challenges such as configuring the replica set and ensuring the correct use of sessions in each operation. These issues were resolved by carefully following the setup steps and verifying the connection and transaction flow.

Transactions are important because they ensure that multiple related operations either all succeed or all fail together. This prevents data inconsistency, such as deducting money from one account without adding it to another. Overall, transactions play a critical role in maintaining reliable and accurate data in database systems.

Conclusion

This lab demonstrated the importance of MongoDB transactions in maintaining data consistency. It highlighted how transactions can commit successful operations or roll back failed ones. Overall, transactions are essential for building reliable database systems.